



INFORME TÉCNICO / SISTEMAS

API v.2

Área	Ingeniería
Elaborado por	Daniel Quiroz
Revisado por	Gerencia
Versión	v2.0
Fecha	12/05/2026

Clasificación: [Confidencial]

Este informe documenta de manera detallada la API del proyecto Ingear. Se cubren arquitectura, configuración, contrato de datos (schemas), modelo de persistencia (SQLAlchemy), endpoints REST (CRUD y relaciones M:N), manejo de errores, consideraciones de integración con el front y recomendaciones para endurecimiento (producción). Esta versión continúa el mismo informe y agrega los cambios funcionales y técnicos incorporados en la API v.2.

1. Alcance funcional (negocio)

La API soporta el flujo principal:

- Gestión de clientes (datos tributarios y de contacto).
- Gestión de oportunidades comerciales vinculadas a clientes.
- Creación y seguimiento de proyectos (estados por áreas).
- Generación y administración de cotizaciones asociadas a proyectos.
- Catálogo de productos (costos, inventario y metadatos).
- Gestión de despachos y asignación de despachos a proyectos.

2. Tecnologías, librerías y estándares

Estándares y convenciones implementadas:

Clave	Descripción
Framework API	FastAPI (ASGI).
Servidor	Uvicorn (modo --reload en desarrollo).
ORM	SQLAlchemy 2.x (DeclarativeBase, SessionLocal).
Validación	Pydantic v2 (schemas Create/Update/Out).
DB Drivers	psycopg (PostgreSQL) o SQLite (para desarrollo rápido).
Autenticación	JWT Bearer, refresh token persistido como hash y cookie HttpOnly.
Autorización	Permisos por vista comercial y roles operativos por área/cargo.
Integración Gmail	Envío de cotizaciones PDF usando service account y delegación.

Integración World Office	Consulta/sincronización de inventario por ODBC.
Tasas FX	TRM BanRep y cálculo EUR/COP con Banco Central Europeo.
Notificaciones	Notificaciones internas por usuario autenticado.

3. Estructura del proyecto (módulos)

Estructura y responsabilidad por capa:

Capa	Ruta	Responsabilidad
Entrada	app/main.py	Crea FastAPI, configura CORS, incluye router /api/v1, endpoint raíz /.
Routing	app/api/v1/api.py	Agrupar routers por recurso (clientes, productos, etc.).
Endpoints	app/api/v1/endpoints/*	Controladores HTTP: validación de entrada, respuestas y errores.
Schemas	app/schemas/*	Contrato de datos (Create/Update/Out) usando Pydantic.

Modelos	app/models/*	Modelo relacional SQLAlchemy: tablas, relaciones, constraints.
CRUD	app/crud/*	Operaciones de BD reutilizables; manejo de IntegrityError → HTTP 409.
DB	app/db/session.py	Engine, SessionLocal, dependencia get_db().
Config	app/core/config.py, .env	APP_NAME, DATABASE_URL vía pydantic-settings.

```

ingear_api/
├── app/
│   ├── main.py
│   ├── core/
│   │   └── config.py
│   ├── db/
│   │   ├── base.py
│   │   └── session.py
│   ├── models/           # Modelos SQLAlchemy (tablas)
│   ├── schemas/          # Modelos Pydantic (request/response)
│   ├── crud/             # CRUD base y CRUD por entidad
│   └── api/v1/
│       ├── api.py        # Enrutador principal
│       └── endpoints/    # Rutas por entidad + relaciones del proyecto

```

4. Configuración y variables de entorno

Crea un archivo .env en la raíz del proyecto (junto a requirements.txt). Ejemplos:

- **Ejemplo con PostgreSQL**

```

DATABASE_URL=postgresql+psycopg://USUARIO:CLAVE@localhost:5432/ingear_db
APP_NAME=Ingear API

```

- **Ejemplo con SQLite (rápido para desarrollo)**

```
DATABASE_URL=sqlite:///./ingear.db  
APP_NAME=Ingear API
```

- **Comandos de ejecución (local)**

Instalar dependencias

```
python -m pip install -r requirements.txt
```

- Crear y activar venv.
- `pip install -r requirements.txt`
- Configurar `DATABASE_URL` en `.env` (PostgreSQL).
- `uvicorn app.main:app --reload --host 127.0.0.1 --port 8000`
- Abrir documentación: <http://127.0.0.1:8000/docs>

CORS (integración con front Vite/React)

La API habilita CORS para orígenes de desarrollo típicos (localhost/127.0.0.1 en puertos 5173 y 5174). Se permiten métodos y headers arbitrarios para facilitar consumo desde el front.

Notas importantes

- En Windows, asegúrate de usar barras normales en `DATABASE_URL` (no rutas de archivo).
- Si usas PostgreSQL, verifica que el servicio esté corriendo y que el puerto 5432 esté abierto.
- Si cambias de PostgreSQL a SQLite (o viceversa), elimina/crea la DB según corresponda para evitar inconsistencias.

5. Base de datos y modelo de datos

El proyecto usa SQLAlchemy con modelos declarativos. En desarrollo se crean tablas automáticamente al iniciar la aplicación (`Base.metadata.create_all`). En producción se recomienda usar migraciones con Alembic.

5.1. Tablas principales

- cliente

- empleado
- oportunidad
- proyecto
- cotizacion
- producto
- despachos

5.2. Tablas puente (relaciones M:N)

- proyecto_empleado (proyecto <-> empleado)
- proyecto_cliente (proyecto <-> cliente)
- proyecto_despacho (proyecto <-> despacho)

5.3. Relaciones relevantes (resumen)

- Cliente 1:N Oportunidad (cliente.oportunidades).
- Empleado 1:N Cotizacion (empleado.cotizaciones).
- Proyecto 1:N Cotizacion (proyecto.cotizaciones).
- Oportunidad 1:1 (opcional) Proyecto (proyecto.oportunidad_id).
- Proyecto M:N Empleado / Cliente / Despacho vía tablas puente.

5.4. Restricciones y unicidad

- cliente.nit es único (uq_cliente_nit).
- empleado.cedula es único (uq_empleado_cedula).
- cotizacion.consecutivo es único (uq_cotizacion_consecutivo).
- producto.codigo_producto es único (uq_producto_codigo).

5.5. Actualización v2: integración World Office

La advertencia de la versión 1 queda actualizada: el backend actual no depende de una tabla world_oficce para crear el esquema base. La integración con World Office se administra por configuración ODBC y servicios de sincronización de inventario.

5.6. Diccionario de datos (campos principales)

Cliente

Campo	Tipo	Notas
id	int	PK
razon_social	string(255)	Requerido
nit	string(50)	Único; index
telefono	string(50)	Opcional
email	EmailStr	Opcional (validación Pydantic)

direccion	string(255)	Opcional
ciudad	string(120)	Opcional
world_office_id	string(80)	FK -> world_oficce.id (requerido en el modelo actual)

Empleado

Campo	Tipo	Notas
id	int	PK
nombre	string(200)	Requerido
email	EmailStr	Opcional
cargo	string(120)	Opcional
area	string(120)	Opcional
estado	string(50)	Opcional
cedula	string(50)	Único; index
telefono	string(50)	Opcional

Oportunidad

Campo	Tipo	Notas
id	int	PK
cliente_id	int	FK -> cliente.id
responsable_cotizacion	int	FK -> empleado.id
fecha_oportunidad	date	Requerido en el modelo
fecha_cotizacion	date	Opcional
porcentaje_cierre	numeric(5,2)	Opcional
fecha_cierre	date	Opcional

Proyecto

Campo	Tipo	Notas
id	int	PK
nombre	string(255)	Requerido
url_cuenta_cobro	string(500)	Opcional
anticipo	numeric(14,2)	Opcional
fecha_inicio	date	Opcional
estado_logistica/contable/in genieria/factura	string(80)	Opcional

observacion	text	Opcional
oportunidad_id	int	FK -> oportunidad.id (opcional)

Cotizacion

Campo	Tipo	Notas
id	int	PK
consecutivo	string(60)	Único; index
id_cotizador	int	FK -> empleado.id
id_proyecto	int	FK -> proyecto.id
url_cotizacion	string(500)	Opcional
tiempo_entrega	string(120)	Opcional
fecha_creacion	datetime	server_default=now()
total	numeric(14,2)	Opcional

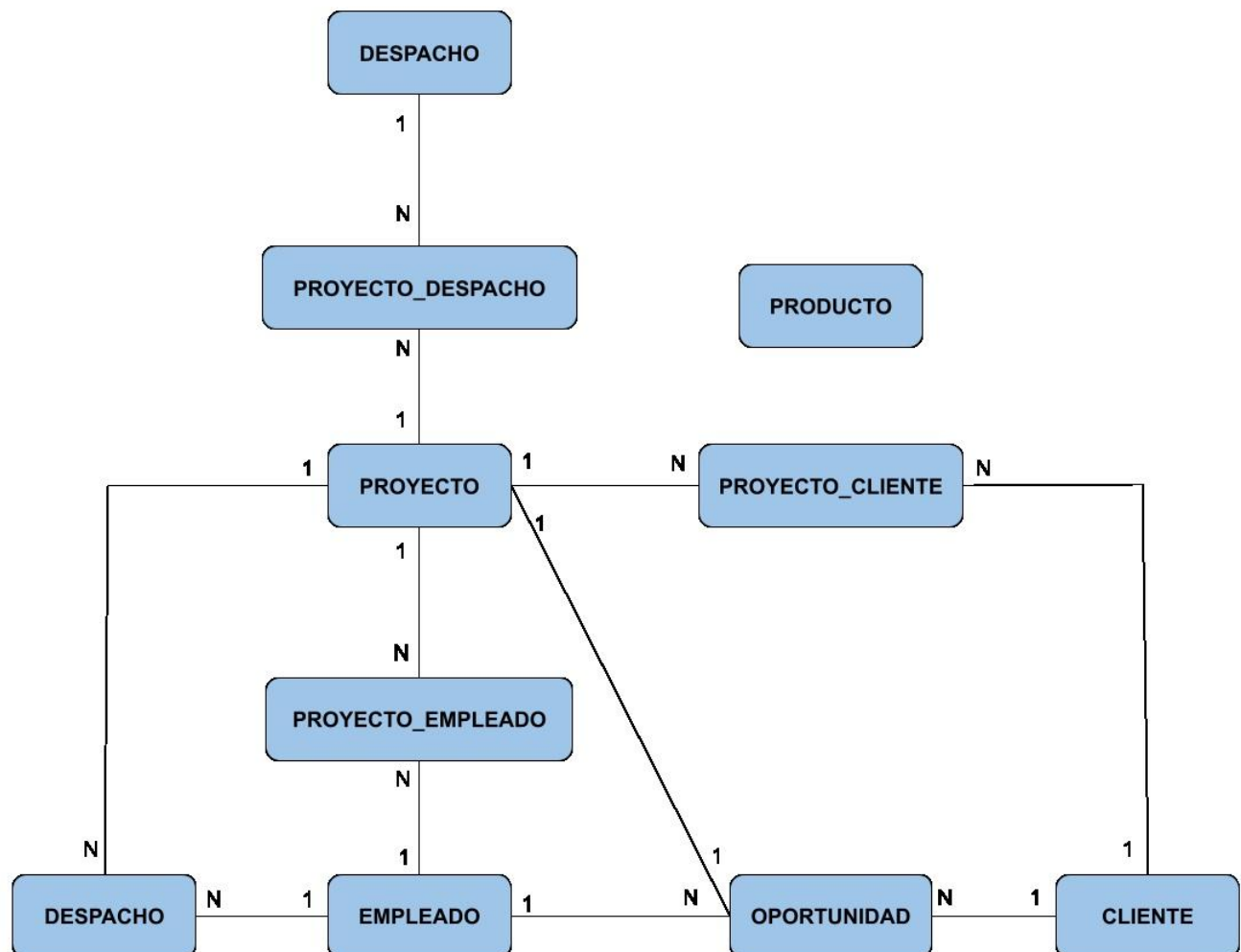
Producto

Campo	Tipo	Notas
id	int	PK
codigo_producto	string(80)	Único; index
marca	string(120)	Opcional
descripcion	string(500)	Opcional
cantidad_inventario	int	Default=0
url_imagen / url_ficha_tecnica	string(500)	Opcional

Despacho

Campo	Tipo	Notas
id	int	PK
fecha	date	Requerido en el modelo
transportadora	string(120)	Opcional
guia	string(120)	Opcional
valor_flete_seguro	numeric(14,2)	Opcional
ciudad_origen / ciudad_destino	string(120)	Opcional

estado	string(80)	Opcional
--------	------------	----------



7. Endpoints (API v1)

Prefijo base de la API: /api/v1

7.1. Endpoints CRUD por entidad

Todas las entidades principales siguen el mismo patrón de rutas: GET (lista), GET (por id), POST (crear), PUT (actualizar), DELETE (eliminar). La lista soporta paginación por query params: skip y limit.

Recurso	Base path	Schemas
Clientes	/api/v1/clientes	ClienteCreate / ClienteUpdate / ClienteOut
Empleados	/api/v1/empleados	EmpleadoCreate / EmpleadoUpdate / EmpleadoOut
Oportunidades	/api/v1/oportunidades	OportunidadCreate / OportunidadUpdate / OportunidadOut
Cotizaciones	/api/v1/cotizaciones	CotizacionCreate / CotizacionUpdate / CotizacionOut
Proyectos	/api/v1/proyectos	ProyectoCreate / ProyectoUpdate / ProyectoOut
Productos	/api/v1/productos	ProductoCreate / ProductoUpdate / ProductoOut
Despachos	/api/v1/despachos	DespachoCreate / DespachoUpdate / DespachoOut

Ejemplos (clientes):

```
# Listar
curl http://127.0.0.1:8000/api/v1/clientes?skip=0&limit=50

# Obtener por id
curl http://127.0.0.1:8000/api/v1/clientes/1

# Crear
curl -X POST http://127.0.0.1:8000/api/v1/clientes ^
  -H "Content-Type: application/json" ^
  -d '{"razon_social":"ACME","nit":"900123","world_office_id":"WO-001"}'
```

7.2. Relaciones del proyecto (M:N)

Las relaciones se gestionan bajo el prefijo `/api/v1/proyectos/{proyecto_id}/...` y operan sobre tablas puente. Al asignar, si la relación ya existe se responde 409.

Relación	Asignar (POST)	Quitar (DELETE)	Body (POST)
Proyecto - Empleado	<code>/api/v1/proyectos/{proyecto_id}/empleados</code>	<code>/api/v1/proyectos/{proyecto_id}/empleados/{empleado_id}</code>	<code>{ "id_empleado": 1 }</code>
Proyecto - Cliente	<code>/api/v1/proyectos/{proyecto_id}/clientes</code>	<code>/api/v1/proyectos/{proyecto_id}/clientes/{cliente_id}</code>	<code>{ "id_cliente": 10 }</code>
Proyecto - Despacho	<code>/api/v1/proyectos/{proyecto_id}/despachos</code>	<code>/api/v1/proyectos/{proyecto_id}/despachos/{despacho_id}</code>	<code>{ "id_despacho": 5 }</code>

Ejemplo asignación empleado a proyecto:

```
curl -X POST http://127.0.0.1:8000/api/v1/proyectos/1/empleados ^
-H "Content-Type: application/json" ^
-d '{"id_empleado": 3}'
```

8. Validación, errores y códigos de respuesta

8.1. Códigos de respuesta típicos

- 200 OK: lectura/actualización exitosa.
- 201 Created: creación exitosa (POST).
- 204 No Content: eliminación exitosa (DELETE).
- 404 Not Found: recurso no existe (mensajes: 'Cliente no encontrado', etc.).
- 409 Conflict: relación duplicada (proyecto-empleado / proyecto-cliente / proyecto-despacho) o colisión de unicidad en DB.
- 422 Unprocessable Entity: validación Pydantic fallida (faltan campos requeridos o tipos incorrectos).

8.2. Validación con Pydantic

Cada recurso tiene tres esquemas Pydantic:

- ***Create***: campos requeridos para crear.
- ***Update***: campos opcionales (`exclude_unset=True`) para actualizar parcialmente.
- ***Out***: modelo de respuesta con id (y campos de auditoría si aplica).

8.3. Paginación

Los endpoints de listado soportan parámetros query:

- skip (int): offset (por defecto 0)
- limit (int): máximo de registros (por defecto 50)

Ejemplo: GET /api/v1/productos?skip=100&limit=50

9. Buenas prácticas y recomendaciones

9.1. Migraciones con Alembic

Actualmente el proyecto crea tablas automáticamente al iniciar. Esto es útil en desarrollo, pero en producción es mejor controlar cambios del esquema con migraciones.

Recomendación:

- 1) Integrar Alembic.
- 2) Desactivar create_all en producción.
- 3) Ejecutar migraciones en el pipeline de despliegue.

9.2. Seguridad

- La versión 2 ya incorpora autenticación JWT, refresh token en cookie HttpOnly, logout, cambio de contraseña y control de sesiones.
- Mantener CORS controlado por entorno, especialmente al pasar de localhost a dominio de producción.
- Registrar auditoría (quién cambia qué) para operaciones críticas.

9.3. Calidad de código y pruebas

- Agregar pruebas con pytest (unitarias para CRUD y de integración para endpoints).
- Separar configuración por entornos (dev/staging/prod).
- Definir linters (ruff/flake8) y formateo (black) para consistencia.

10. Publicación en GitHub (flujo recomendado)

Para subir el proyecto al repositorio, necesitas tener Git instalado y accesible desde la terminal. Si al ejecutar 'git status' aparece 'git no se reconoce', instala Git for Windows y reinicia la terminal.

10.1. Instalación y verificación de Git

```
# Verificar instalación  
git --version
```

```
# Configurar identidad (una sola vez)
git config --global user.name "Tu Nombre"
git config --global user.email "tuemail@dominio.com"
```

10.2. Subir proyecto al repo existente

Opción A: clonar el repo y copiar tu código dentro (recomendado si el repo ya existe).

```
cd D:\
git clone https://github.com/cripto26/api_ingear.git
cd api_ingear
```

```
# Copia aquí el contenido de D:\API_Ingear (excepto .venv)
# Luego:
git add .
git commit -m "Initial commit: Ingear API (FastAPI + SQLAlchemy)"
git push origin main
```

Opción B: inicializar Git en tu carpeta actual y enlazar al remoto (si no has clonado).

```
cd D:\API_Ingear
git init
git branch -M main
git remote add origin https://github.com/cripto26/api_ingear.git
```

```
# Recomendado: crear .gitignore (ver sección 10.3)
git add .
git commit -m "Initial commit"
git push -u origin main
```

10.3. .gitignore recomendado

```
# Python
__pycache__/
*.pyc
*.pyo
*.pyd
.venv/
env/
venv/

# VS Code / IDE
.vscode/

# DB local
*.db
```

```
# Logs
```

```
*.log
```

```
# Env
```

```
.env
```

11. Solución de problemas (troubleshooting)

11.1. ERR_CONNECTION_REFUSED en /docs

Causa típica: el proceso de Uvicorn se cae durante startup. Revisa la consola donde ejecutaste uvicorn y corrige el error antes de volver a intentar.

11.2. Errores de conexión a PostgreSQL

Síntomas comunes: ConnectionTimeout, getaddrinfo failed, o Test-NetConnection al puerto 5432 falla.

- Verifica que PostgreSQL esté instalado y el servicio esté iniciado.
- Confirma usuario/clave/DB en DATABASE_URL.
- Asegura que el puerto 5432 esté escuchando (y no bloqueado por firewall).

11.3. NoReferencedTableError (ForeignKey)

Si ves un error como 'could not find table ... with which to generate a foreign key', hay una FK apuntando a una tabla que no existe o está mal escrita. En esta base, el caso más común es Cliente.world_office_id -> world_oficce.id.

11.4. Conflictos 409 en relaciones

Al asignar un empleado/cliente/despacho a un proyecto, si la relación ya existe se devuelve 409. Solución: no repetir asignación, o eliminar primero la relación y volver a crearla.

12. Actualización técnica API v.2

Esta sección se agrega como continuación directa del informe v1. Mantiene el mismo formato del documento original y consolida los cambios detectados en el backend actual.

12.1. Cambios funcionales incorporados

Autenticación completa con login, refresh, logout, usuario actual y cambio de contraseña.
Control de permisos por vista comercial: cotizador, oportunidades, proyectos, cuentas de cobro, clientes, contactos y productos.

Versionamiento histórico de cotizaciones antes de cada actualización.

Flujo de aprobación de cotizaciones por jefe directo o Gerencia.

Envío de cotizaciones por correo mediante Gmail API.

Precarga de cuentas de cobro desde proyecto, oportunidad, cliente y última cotización.

Notificaciones internas por usuario autenticado.

Consulta de tasas de cambio USD/COP y EUR/COP.

Sincronización de existencias desde World Office.

12.2. Módulos y endpoints agregados o ampliados

Módulo	Prefijo	Métodos	Cambio documentado
auth	/api/v1/auth	GET, POST	Login, refresh, logout, usuario actual y cambio de contraseña.
cotizaciones	/api/v1/cotizaciones	DELETE, GET, POST, PUT	Versionamiento, aprobación, envío por correo y creación de proyecto.
cuentas-cobro	/api/v1/cuentas-cobro	DELETE, GET, POST, PUT	Gestión y precarga desde proyecto, oportunidad, cliente y cotización.
fx	/api/v1/fx	GET	Consulta de tasas USD/COP y EUR/COP con BanRep y BCE.
notificaciones	/api/v1/notificaciones	GET, POST	Listado, conteo y marcado de notificaciones por usuario.
productos	/api/v1/productos	DELETE, GET, POST, PUT	Catálogo, imagen, costos e inventario World Office.
proyectos-relaciones	/api/v1/proyectos	DELETE, POST	Relaciones M:N de proyectos con clientes, despachos y empleados.

12.3. Seguridad y sesiones

El acceso protegido se realiza con token Bearer. La sesión persistente utiliza refresh token en cookie HttpOnly y una tabla auth_refresh_session con hash del token, fechas de expiración, último uso y revocación.

El login puede validar Cloudflare Turnstile y aplica protección contra fuerza bruta mediante límites por usuario/IP, ventana de fallos y bloqueo temporal.

12.4. Cambios en persistencia

Se agregan tablas para auth_refresh_session, cuenta_cobro y notificacion.

Se conserva el modelo principal cliente, contacto, oportunidad, cotizacion, proyecto, producto, despacho y empleado.

Se mantiene el uso de tablas puente proyecto_cliente, proyecto_empleado y proyecto_despacho.

Se agrega historial de cotización en cotizacion_versiones_v2.

Se actualizan columnas de empleado para permisos_vistas y jefe_id cuando el esquema lo requiere.

12.5. Variables de entorno nuevas o relevantes

Variable	Uso
SECRET_KEY	Firma de access tokens JWT.
GOOGLE_SERVICE_ACCOUNT_FILE	Ruta de credenciales para Gmail/Google APIs.
TURNSTILE_SECRET_KEY	Validación opcional de Turnstile en login.
WORLD_OFFICE_ENABLED	Activa la consulta/sincronización de inventario.
WORLD_OFFICE_ODBC_DRIVER	Driver ODBC instalado en el servidor.
LOGIN_FAILURE_LOCK_THRESHOLD	Número de fallos requeridos para bloquear.

12.6. Recomendaciones actualizadas

Usar HTTPS y cookies Secure/SameSite en producción.

Proteger SECRET_KEY, credenciales Google, Turnstile y World Office fuera del repositorio.

Agregar migraciones formales con Alembic para cambios de esquema.

Registrar auditoría de acciones críticas: aprobación, envío de cotización, cambio de estado y cuentas de cobro.

Agregar pruebas de endpoints para autenticación, permisos, cotizaciones, cuentas de cobro e integraciones simuladas.